Enterprise Data Modeling

> **Document Owner:** Platform Architecture Team | **Status:** Approved | **Last Updated:** February 2026

Overview

This document establishes the enterprise data modeling standards for our Databricks Lakehouse built on Unity Catalog and Delta Lake. It defines the patterns, constraints, and best practices that ensure optimal query performance, data integrity, and maintainability across all analytical workloads.

Golden Rules

ID	Rule	Severity	WAF Pillar
DM-01	Use dimensional modeling (star/snowflake schema) for Gold layer	Critical	Performance
DM-02	Declare PRIMARY KEY constraints on all dimension and fact tables	Critical	Governance
DM-03	Declare FOREIGN KEY constraints to document relationships	Required	Governance
DM-04	Avoid heavily normalized models (3NF) in analytical layers	Required	Performance
DM-05	Design for minimal joins in common query patterns	Required	Reliability
DM-06	Use enforced constraints (NOT NULL, CHECK) for data integrity	Required	Performance
DM-07	Handle semi-structured data with appropriate complex types	Recommended	Governance
DM-08	Design for single-table transactions	Critical	Performance
DM-09	Design Gold layer for both BI and AI consumption	Required	Interop

---

DM-01: Dimensional Modeling

Rule

All Gold layer data models must use dimensional modeling patterns (star schema or snowflake schema) rather than heavily normalized relational models.

Why It Matters

- **Query Performance:** Star schemas require fewer joins, improving query speed
- **Data Skipping:** More columns per table enables better file-level statistics and data skipping
- **BI Compatibility:** BI tools and semantic layers are optimized for dimensional models
- **Business Alignment:** Dimensions and facts align with business concepts

Implementation

Star Schema Pattern (Preferred):

```
                ┌─────────────────┐
                │   dim_date      │
                └─────────────────┘
                         │
                         │
┌─────────────────┐      │      ┌─────────────────┐
│ dim_customer    ├──────┼──────┤ dim_product     │
└─────────────────┘      │      └─────────────────┘
                         │
                         │
                ┌─────────────────┐
                │ fact_sales      │
                └─────────────────┘
```

Gold Layer ERD Example:

```sql
-- Fact table with foreign keys to dimensions
CREATE TABLE gold.fact_sales (
    sale_id BIGINT NOT NULL,
    sale_date DATE NOT NULL,
    customer_key STRING NOT NULL,
    product_key STRING NOT NULL,
    store_key STRING NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(18,2) NOT NULL,
    total_amount DECIMAL(18,2) NOT NULL,
    CONSTRAINT pk_fact_sales PRIMARY KEY (sale_id) NOT ENFORCED,
    CONSTRAINT fk_customer FOREIGN KEY (customer_key)
        REFERENCES gold.dim_customer(customer_key) NOT ENFORCED,
    CONSTRAINT fk_product FOREIGN KEY (product_key)
        REFERENCES gold.dim_product(product_key) NOT ENFORCED
)
USING DELTA
CLUSTER BY AUTO;
```

Anti-Pattern: Avoid 3NF in Gold

Pattern	Gold Layer	Why
❌ Third Normal Form (3NF)	Not recommended	Too many joins, poor query performance

Pattern	Gold Layer	Why
✅ Star Schema	Recommended	Optimal balance of performance and normalization
✅ Snowflake Schema	Acceptable	When dimension hierarchies are needed
✅ Denormalized	Acceptable	For specific performance-critical queries

DM-02: Primary Key Constraints

Rule

All Gold layer dimension and fact tables must declare PRIMARY KEY constraints to establish entity identity and enable query optimization.

Why It Matters

- **Query Optimization:** Databricks uses PK/FK information for join ordering and optimization
- **Data Quality:** Documents uniqueness requirements for each table
- **Semantic Layer:** Metric Views and Genie use relationships for query generation
- **Documentation:** Self-documenting schema relationships

Implementation

Dimension Tables (Surrogate Key):

```
-- SCD Type 2 dimension with surrogate key
CREATE TABLE gold.dim_customer (
    customer_key STRING NOT NULL
        COMMENT 'Surrogate key. Technical: MD5 hash of customer_id +
effective_from.',
    customer_id STRING NOT NULL
        COMMENT 'Business key. Stable across SCD2 versions.',
    customer_name STRING,
    effective_from TIMESTAMP NOT NULL,
    effective_to TIMESTAMP,
    is_current BOOLEAN NOT NULL,
    CONSTRAINT pk_dim_customer PRIMARY KEY (customer_key) NOT ENFORCED
)
USING DELTA
CLUSTER BY AUTO;
```

Fact Tables (Composite Key):

```
-- Daily aggregated fact with composite primary key
CREATE TABLE gold.fact_sales_daily (
    sale_date DATE NOT NULL,
    customer_key STRING NOT NULL,
    product_key STRING NOT NULL,
```

```
    store_key STRING NOT NULL,
    total_quantity INT NOT NULL,
    total_revenue DECIMAL(18,2) NOT NULL,
    CONSTRAINT pk_fact_sales_daily
        PRIMARY KEY (sale_date, customer_key, product_key, store_key) NOT
ENFORCED
)
USING DELTA
CLUSTER BY AUTO;
```

Important Notes

Aspect	Databricks Behavior
Enforcement	PRIMARY KEY is **informational only** (NOT ENFORCED)
Uniqueness	Must be enforced via MERGE logic or upstream validation
Column Nullability	PK columns should be NOT NULL
Optimization	Used by query optimizer for join strategies

DM-03: Foreign Key Constraints

Rule

All Gold layer fact tables must declare FOREIGN KEY constraints to document relationships with dimension tables.

Why It Matters

- **Relationship Documentation:** Explicit documentation of data model relationships
- **Query Optimization:** Optimizer uses FK information for efficient join planning
- **Semantic Layer:** Metric Views auto-detect relationships for natural language queries
- **Data Lineage:** Enables automated lineage tracking

Implementation

```
-- Add FK constraints after all tables and PKs exist
ALTER TABLE gold.fact_sales
ADD CONSTRAINT fk_sales_customer
    FOREIGN KEY (customer_key) REFERENCES gold.dim_customer(customer_key)
NOT ENFORCED;

ALTER TABLE gold.fact_sales
ADD CONSTRAINT fk_sales_product
    FOREIGN KEY (product_key) REFERENCES gold.dim_product(product_key) NOT
ENFORCED;

ALTER TABLE gold.fact_sales
```

```
ADD CONSTRAINT fk_sales_date
    FOREIGN KEY (sale_date) REFERENCES gold.dim_date(date_key) NOT
ENFORCED;
```

Constraint Application Order

Critical: Apply constraints in correct order

Step	Action	Why
1	Create all tables	Tables must exist
2	Add all PRIMARY KEYs	PKs must exist before FKs reference them
3	Add FOREIGN KEYs	Now PK targets exist

Common Error:

```
-- ❌ WRONG: FK applied before target PK exists
ALTER TABLE fact_sales ADD CONSTRAINT fk_customer
    FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key);
-- Error: dim_customer does not have a primary key
```

DM-04: Avoid Heavy Normalization

Rule

Avoid heavily normalized models (Third Normal Form and beyond) in analytical Gold layer. Reserve normalization for transactional Bronze/Silver layers.

Why It Matters

- **Join Performance:** Each join operation adds processing overhead
- **Query Complexity:** Complex queries with many joins are harder to optimize
- **Data Skipping:** More columns per table enables better data skipping
- **BI Tool Compatibility:** Most BI tools expect dimensional models

Normalization Comparison

Model	Joins Required	Query Performance	Use Case
3NF (Normalized)	Many (5-10+)	Slower	OLTP systems
Star Schema	Few (2-5)	Fast	Analytical workloads ✅
Snowflake	Moderate (3-7)	Moderate	Hierarchical dimensions
Denormalized	None (1)	Fastest	Specific hot queries

Implementation

❌ **Over-Normalized (Avoid in Gold):**

```
customer → customer_address → city → state → region → country
         → customer_contact → contact_type
         → customer_segment → segment_category
```

Result: 7+ joins for a simple customer query

✅ **Dimensional (Recommended for Gold):**

```
dim_customer (includes flattened address, contact, segment)
    |
fact_sales
    |
dim_product (includes flattened category, brand)
```

Result: 2-3 joins for most queries

When to Denormalize

Scenario	Recommendation
Frequently joined dimensions	Consider denormalizing into fact
Slowly changing hierarchies	Snowflake schema acceptable
Real-time dashboards	Fully denormalized aggregates
Ad-hoc exploration	Star schema with good indexing

DM-05: Minimize Joins in Common Queries

Rule

Design data models to minimize joins for the most common query patterns. Pre-join frequently accessed dimensions into facts when performance requires it.

Why It Matters

- **Query Performance:** Joins can be the biggest bottleneck in query execution
- **Filter Pushdown:** Filters on joined tables may not push down efficiently
- **Full Table Scans:** Poor join conditions can cause full table scans
- **Optimizer Limitations:** Complex multi-table joins challenge the optimizer

Implementation

Identify Common Query Patterns:

```sql
-- Analyze most common join patterns in your workload
SELECT
    CONCAT(source_table, ' → ', target_table) as join_pattern,
    COUNT(*) as frequency
FROM system.query_history
WHERE query_text LIKE '%JOIN%'
GROUP BY 1
ORDER BY 2 DESC
LIMIT 10;
```

Pre-Join Frequently Accessed Attributes:

```sql
-- Instead of always joining for customer segment
-- Include commonly needed attributes in fact
CREATE TABLE gold.fact_sales_enriched (
    sale_date DATE,
    customer_key STRING,
    customer_segment STRING,  -- Denormalized from dim_customer
    customer_region STRING,   -- Denormalized from dim_customer
    product_category STRING,  -- Denormalized from dim_product
    total_amount DECIMAL(18,2)
)
USING DELTA
CLUSTER BY AUTO;
```

Join Optimization Strategies

Strategy	When to Use	Trade-off
Pre-join into fact	High-frequency attributes	Storage increase
Materialized views	Complex aggregations	Refresh latency
Broadcast hints	Small dimension tables	Memory usage
Delta caching	Repeated queries	Compute memory

DM-06: Enforced Constraints

Rule

Use enforced constraints (NOT NULL, CHECK) to maintain data integrity at the table level for critical data quality requirements.

Why It Matters

- **Data Quality:** Prevents invalid data from entering tables

- **Transaction Safety:** Invalid inserts/updates fail immediately
- **Documentation:** Self-documenting data requirements
- **Downstream Trust:** Consumers can rely on data validity

Implementation

NOT NULL Constraints:

```sql
CREATE TABLE gold.fact_sales (
    sale_id BIGINT NOT NULL,          -- Required: primary identifier
    sale_date DATE NOT NULL,          -- Required: time dimension
    customer_key STRING NOT NULL,     -- Required: who bought
    amount DECIMAL(18,2) NOT NULL,    -- Required: core metric
    discount_pct DECIMAL(5,2)         -- Optional: may be null
)
USING DELTA;

-- Modify existing column
ALTER TABLE gold.fact_sales ALTER COLUMN store_key SET NOT NULL;
```

CHECK Constraints:

```sql
-- Add business rule validations
ALTER TABLE gold.fact_sales
ADD CONSTRAINT valid_amount CHECK (amount >= 0);

ALTER TABLE gold.fact_sales
ADD CONSTRAINT valid_discount CHECK (discount_pct BETWEEN 0 AND 100);

ALTER TABLE gold.dim_customer
ADD CONSTRAINT valid_dates CHECK (effective_from <= COALESCE(effective_to,
'9999-12-31'));
```

Enforced vs Informational Constraints

Constraint Type	Enforced	Use Case
NOT NULL	✅ Yes	Required fields
CHECK	✅ Yes	Business rules
PRIMARY KEY	❌ No	Identity documentation
FOREIGN KEY	❌ No	Relationship documentation

DM-07: Semi-Structured Data

Rule

Use appropriate complex data types (STRUCT, ARRAY, MAP) for semi-structured data. Avoid storing raw JSON strings when structured access is needed.

Why It Matters

- **Query Performance:** Native types are faster than parsing JSON strings
- **Schema Enforcement:** Complex types provide schema validation
- **SQL Access:** Direct field access without JSON parsing functions
- **Storage Efficiency:** Binary storage more compact than string JSON

Implementation

STRUCT for Nested Objects:

```sql
CREATE TABLE gold.dim_customer (
    customer_id STRING,
    customer_name STRING,
    address STRUCT<
        street: STRING,
        city: STRING,
        state: STRING,
        zip: STRING,
        country: STRING
    >,
    preferences STRUCT<
        newsletter: BOOLEAN,
        language: STRING,
        currency: STRING
    >
);

-- Query nested fields
SELECT
    customer_name,
    address.city,
    address.state,
    preferences.language
FROM gold.dim_customer;
```

ARRAY for Collections:

```sql
CREATE TABLE gold.dim_product (
    product_id STRING,
    product_name STRING,
    tags ARRAY<STRING>,
    price_history ARRAY<STRUCT<
        effective_date: DATE,
        price: DECIMAL(18,2)
    >>
);
```

```sql
-- Query array elements
SELECT
    product_name,
    tags[0] as primary_tag,
    SIZE(price_history) as price_changes
FROM gold.dim_product;
```

MAP for Dynamic Key-Value:

```sql
CREATE TABLE gold.fact_events (
    event_id STRING,
    event_type STRING,
    event_properties MAP<STRING, STRING>
);

-- Query map values
SELECT
    event_type,
    event_properties['source'] as source,
    event_properties['campaign'] as campaign
FROM gold.fact_events;
```

When to Use Each Type

Type	Use Case	Example
STRUCT	Fixed schema nested objects	Address, preferences
ARRAY	Ordered collections	Tags, history
MAP	Dynamic key-value pairs	Custom properties
JSON STRING	Truly schema-less, rare access	Raw API responses

DM-08: Single-Table Transactions

Rule

Design data pipelines recognizing that Databricks scopes transactions to individual tables. Multi-table atomicity requires application-level handling.

Why It Matters

- **Transaction Scope:** Each table MERGE/INSERT is an independent transaction
- **Consistency:** Multi-table updates may have intermediate inconsistent states
- **Recovery:** Failed multi-table operations require explicit rollback handling
- **Query Timing:** Downstream queries may see partial updates

Implementation

Handle Multi-Table Updates:

```
# Each operation is a separate transaction
try:
    # Transaction 1: Update dimension
    spark.sql("""
        MERGE INTO gold.dim_customer AS target
        USING staging.customer_updates AS source
        ON target.customer_id = source.customer_id
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)

    # Transaction 2: Update fact (separate transaction)
    spark.sql("""
        MERGE INTO gold.fact_sales AS target
        USING staging.sales_updates AS source
        ON target.sale_id = source.sale_id
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)

except Exception as e:
    # Application-level error handling
    # Consider: logging, alerting, manual review
    raise
```

Design for Eventual Consistency:

Pattern	Description	Use Case
Dimension-First	Update dimensions before facts	FK validity
Idempotent Operations	Design for safe retry	Recovery
Checkpoint Tables	Track completed operations	Resume points
Versioned Updates	Include version/timestamp	Conflict detection

Query Tolerance:

```
-- Design queries to handle eventual consistency
-- Use COALESCE for potentially missing dimension data
SELECT
    f.sale_date,
    COALESCE(d.customer_name, 'Unknown') as customer_name,
    f.amount
FROM gold.fact_sales f
LEFT JOIN gold.dim_customer d ON f.customer_key = d.customer_key;
```

DM-09: Design for BI and AI Reuse

Rule

Design Gold layer data models to serve both BI (dashboards, Genie, Metric Views) and AI (feature engineering, model training, agent grounding) workloads without requiring separate pipelines.

Why It Matters

- **Eliminates duplicate pipelines:** ML teams often rebuild transformations that already exist in Gold
- **Ensures consistency:** BI and AI use the same governed data
- **Reduces time-to-model:** Feature tables derived from Gold are immediately production-quality
- **Simplifies lineage:** Single path from source to consumption

Implementation

Design Gold dimensions with ML-friendly attributes:

```sql
-- Gold dimension designed for both BI queries and ML feature extraction
CREATE TABLE gold.dim_customer (
    customer_key STRING NOT NULL,
    customer_id STRING NOT NULL,
    customer_name STRING,
    -- BI-friendly attributes
    customer_segment STRING COMMENT 'Business segment. Used in dashboards
and Genie.',
    region STRING,
    -- ML-friendly numeric attributes (clean, no NaN)
    lifetime_value DECIMAL(18,2) NOT NULL DEFAULT 0
        COMMENT 'Total spend in USD. ML: numeric feature, no nulls.',
    order_count INT NOT NULL DEFAULT 0
        COMMENT 'Total orders. ML: numeric feature, no nulls.',
    days_since_last_order INT
        COMMENT 'Recency metric. ML: numeric feature for churn
prediction.',
    -- Metadata
    effective_from TIMESTAMP NOT NULL,
    effective_to TIMESTAMP,
    is_current BOOLEAN NOT NULL,
    CONSTRAINT pk_dim_customer PRIMARY KEY (customer_key) NOT ENFORCED
)
USING DELTA
CLUSTER BY AUTO;
```

Design Principles for Dual Use

Principle	BI Benefit	AI Benefit

Principle	BI Benefit	AI Benefit
NOT NULL on numeric columns	Clean aggregations	No NaN handling needed at training time
DEFAULT values on metrics	Predictable zero-state	Feature tables ready without imputation
Clear column COMMENTs	Genie understands context	Feature discovery for ML engineers
PK/FK constraints	Metric View relationships	Feature table join keys
`is_current` flag on SCD2	Simple current-state queries	Point-in-time feature extraction

Data Model Documentation

Required Documentation

Every data domain must include:

Artifact	Description	Format
ERD Diagram	Visual relationship map	Mermaid or draw.io
YAML Schemas	Column definitions, types, constraints	YAML files
Grain Statement	What each row represents	Documentation
Change Strategy	SCD type, update frequency	Documentation

ERD Example (Mermaid)

```
erDiagram
    dim_customer ||--o{ fact_sales : "customer_key"
    dim_product ||--o{ fact_sales : "product_key"
    dim_date ||--o{ fact_sales : "sale_date"
    dim_store ||--o{ fact_sales : "store_key"

    dim_customer {
        string customer_key PK
        string customer_id BK
        string customer_name
        boolean is_current
    }

    fact_sales {
        bigint sale_id PK
        date sale_date FK
        string customer_key FK
        string product_key FK
        decimal amount
    }
```

Validation Checklist

Before deploying a new data model:

Design Phase

- [ ] Dimensional model designed (star or snowflake schema)
- [ ] ERD diagram created and reviewed
- [ ] Grain documented for all fact tables
- [ ] SCD strategy defined for dimensions

Implementation Phase

- [ ] All tables use `CLUSTER BY AUTO`
- [ ] All dimensions have PRIMARY KEY constraint
- [ ] All facts have composite PRIMARY KEY constraint
- [ ] FOREIGN KEY constraints applied (after PKs)
- [ ] NOT NULL on required columns
- [ ] CHECK constraints for business rules

Documentation Phase

- [ ] Table COMMENTs with LLM-friendly descriptions
- [ ] Column COMMENTs with business + technical context
- [ ] YAML schema files created
- [ ] Data lineage documented

Related Documents

- Data Governance
- Naming & Comment Standards
- Tagging Standards
- Data Quality Standards
- Unity Catalog Tables
- Gold Layer Patterns
- Architecture Review Checklist

References

- Constraints on Azure Databricks
- Data Modeling on Azure Databricks
- Delta Lake Primary and Foreign Keys
- Star Schema Best Practices